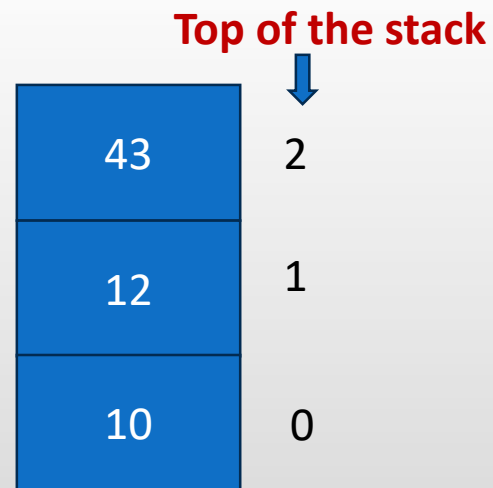


Introduction

Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO (Last In First Out) or FILO (First In Last Out).

LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.



Real-Life Applications of Stack

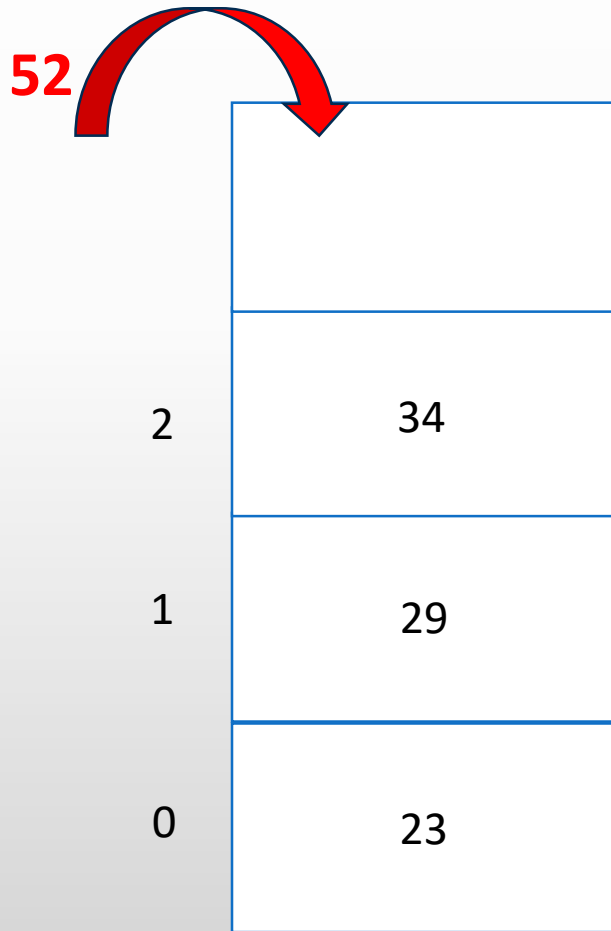


Operations on Stack

- **push()**: When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop()**: When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state
- **isEmpty()**: It determines whether the stack is empty or not.
- **isFull()**: It determines whether the stack is full or not.'
- **display()**: It prints all the elements available in the stack.
- **peek()**: It returns the element at the given position.
- **count()**: It returns the total number of elements available in a stack.

Push Operation

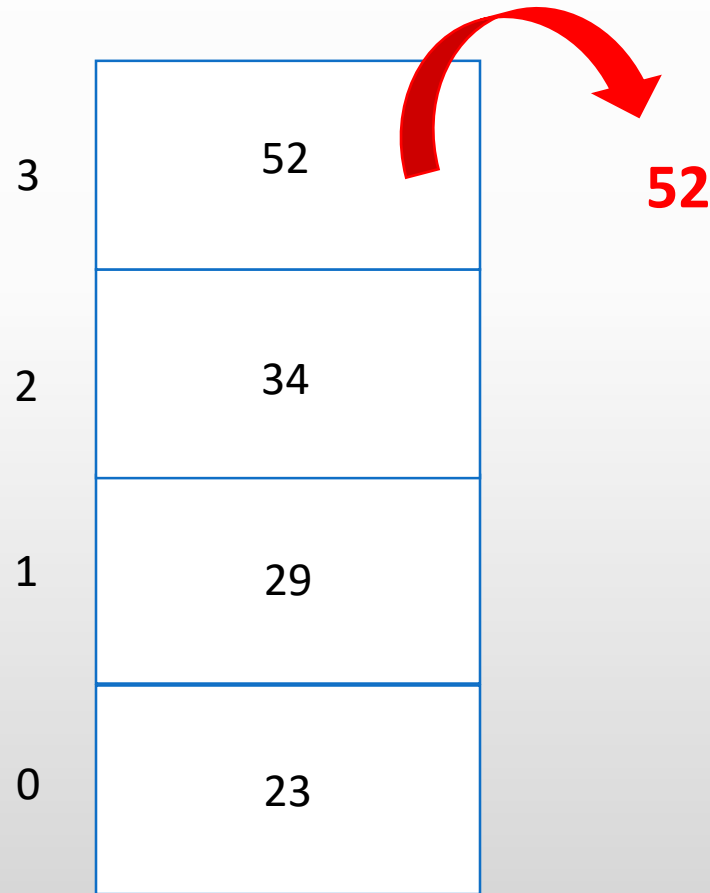
It stacks up a new item. A stack overflow circumstance is when the stack is completely full.



```
void push ()
{
    int num;
    if (s.top == (MAXSIZE - 1))
    {
        printf ("Stack is Full\n");
        return;
    }
    else
    {
        printf ("Enter the element to push\n");
        scanf ("%d", &num);
        s.top = s.top + 1;
        s.stk[s.top] = num;
    }
    return;
}
```

POP Operation

It takes something out of the stack. In the opposite sequence from which they were pushed, the things are popped. The condition is referred to as an underflow if the stack is empty.



```
int pop ()
{
    int num;
    if (s.top == - 1)
    {
        printf ("Stack is Empty\n");
        return (s.top);
    }
    else
    {
        num = s.stk[s.top];
        printf ("poped element is = %dn", s.stk[s.top]);
        s.top = s.top - 1;
    }
    return(num);
}
```

Array Implementation of Stack

In array implementation, the stack is formed by using the array. All the operations regarding the stack are performed using arrays.

Adding an element onto the stack (push operation)

Adding an element into the top of the stack is referred to as push operation. Push operation involves following two steps.

1. Increment the variable Top so that it can now refer to the next memory location.
2. Add element at the position of incremented top. This is referred to as adding new element at the top of the stack.

Stack is overflown when we try to insert an element into a completely filled stack therefore, our main function must always avoid stack overflow condition.

Deletion of an element from a stack (Pop operation)

Deletion of an element from the top of the stack is called pop operation. The value of the variable top will be incremented by 1 whenever an item is deleted from the stack. The top most element of the stack is stored in an another variable and then the top is decremented by 1. the operation returns the deleted value that was stored in another variable as the result.

The underflow condition occurs when we try to delete an element from an already empty stack.

Various Stack Types

- **Register Stack:** The memory unit also has a form of stack called a register that can only handle a tiny amount of data. Because the capacity of the register stack is so little in comparison to the memory, its height is constantly constrained.
- **Memory Stack:** This kind of stack is capable of managing a lot of data from memory. The memory stack can be any height because it uses a lot of memory space.

Applications of Stack

- Conversion from Infix to Postfix or Prefix
- There are redo-undo options in numerous locations, including editors like Photoshop.
- Web browsers have forward and backward functions.
- Computers' implementation of function calls also benefits from stack. The function that was last invoked is always finished first.

Thank You